

Lab#06

Department of Computer Science (7th Semester)

Course: DevOps

Name:

Zahra Bashir

SUBMITTED TO:

Sir Waqas Saleem

Registration Number

23-BCS-092(Section-A)

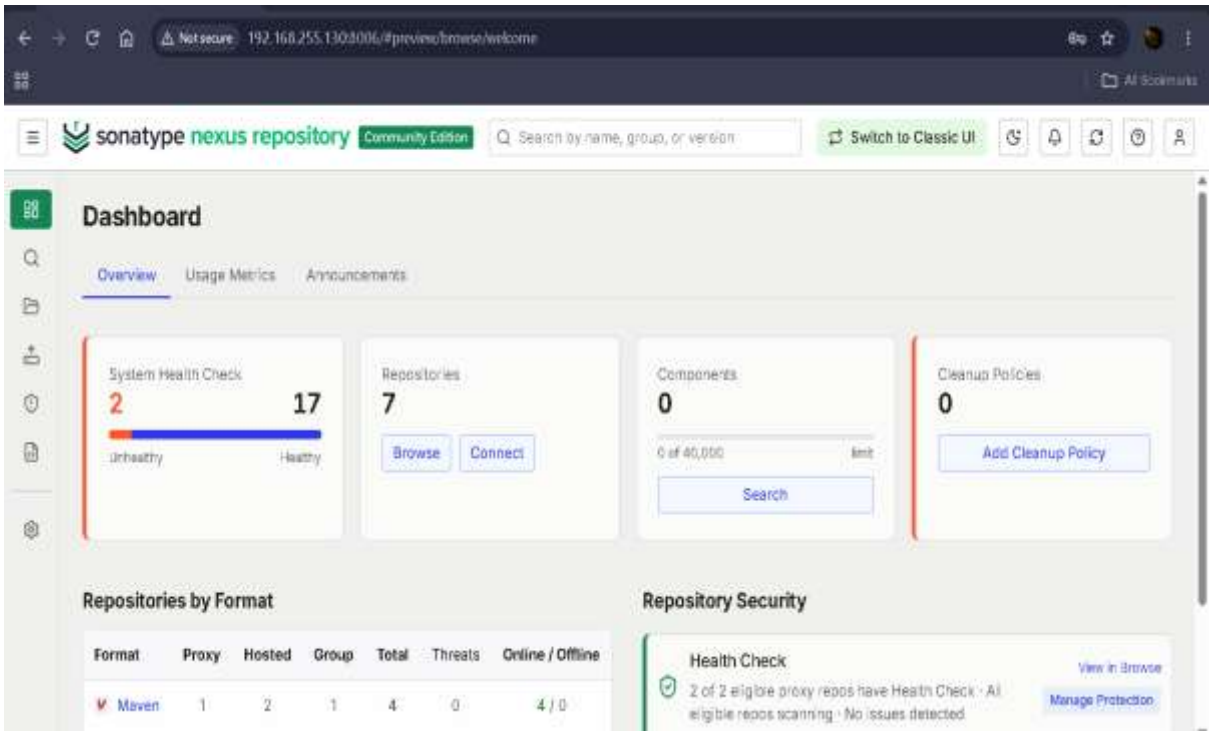
DevOps Repository Link: https://github.com/zahra225/DevOps

Submission: DevOps / Labs / Lab06.

Task 1: Run Nexus Repository and Complete the Initial Setup

I started the Nexus container on the Ubuntu server with persistent storage (nexus-data volume) and published ports 8006 and 8007. I opened the Nexus web interface at http://<ubuntu-server-ip>:8006, signed in as admin with the one-time password, set a new admin password and disabled anonymous access in the setup wizard.

Screenshot 1: nexus_container_running.png

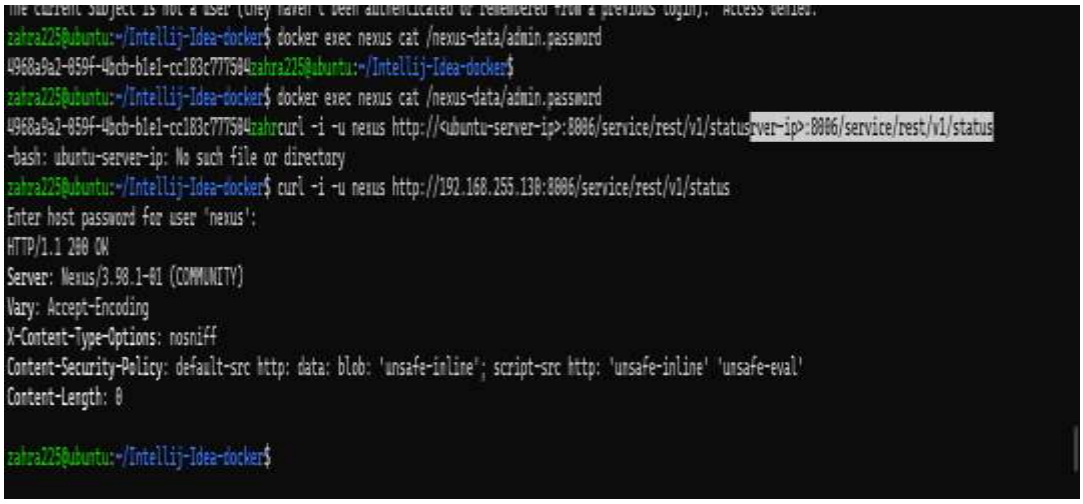


docker ps | grep nexus shows the nexus container running with ports 8006 and 8007 published.

Task 2: Create a Role and User and Verify Access

I created the nx-java role with the nx-repository-view-maven2-maven-snapshots-* privilege and created the active local user nexus with the nx-java role granted. I verified the user's credentials through the Nexus Status API with curl.

Screenshot 2: nexus_user_access.png

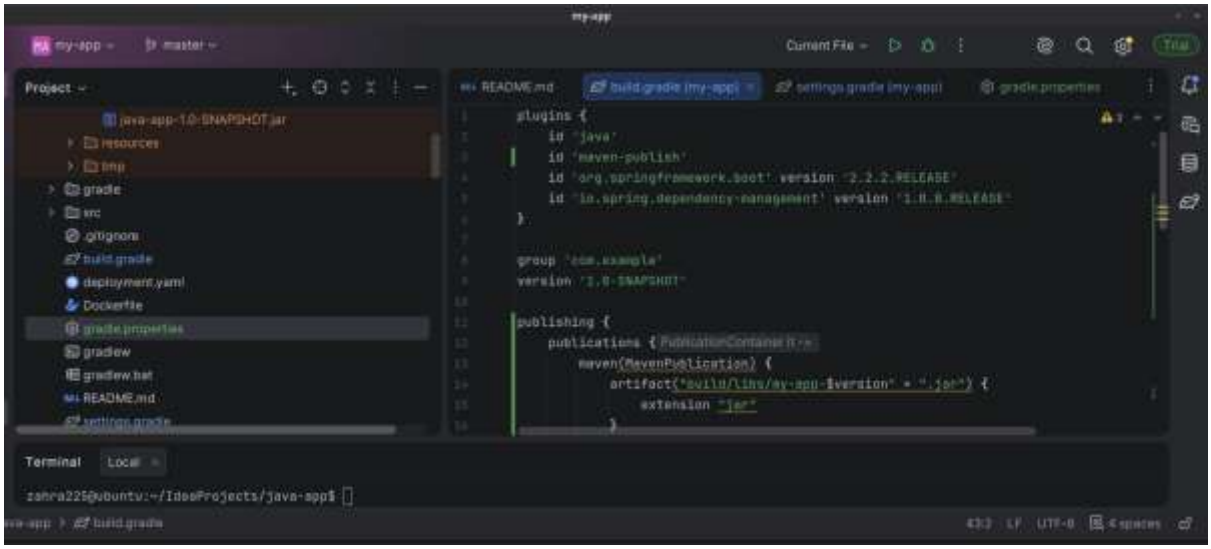


curl -i -u nexus .../service/rest/v1/status returned HTTP/1.1 200 OK; the password is not visible.

Task 3: Build and Publish the Gradle Application to Nexus

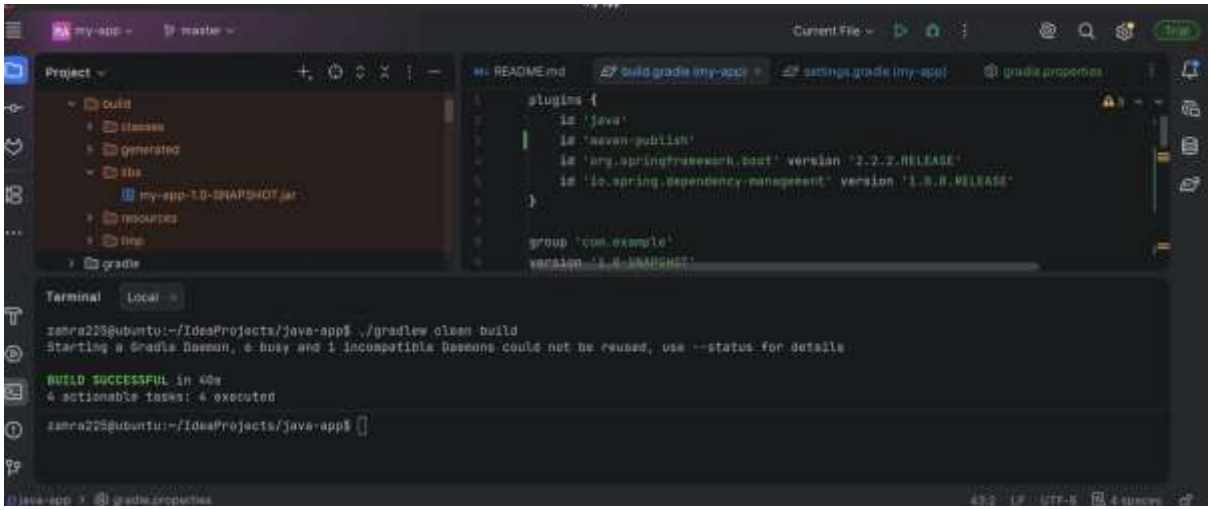
I opened the java-app Gradle project with JDK 16, added the maven-publish plugin and a publishing block to build.gradle, set rootProject.name to my-app in settings.gradle and stored the Nexus credentials in gradle.properties. I built my-app-1.0-SNAPSHOT.jar, published it with ./gradlew clean build publish and verified it in the maven-snapshots repository.

Screenshot 3: gradle_nexus_configuration.png



Updated build.gradle publishing configuration and settings.gradle (credentials not shown).

Screenshot 4: gradle_nexus_build.png



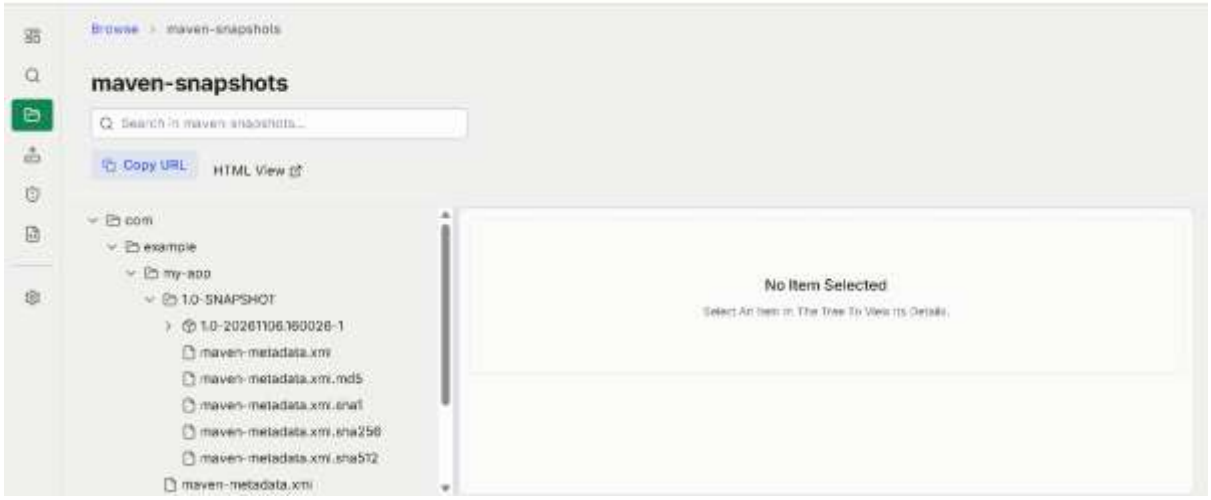
./gradlew clean build finished with BUILD SUCCESSFUL and created my-app-1.0-SNAPSHOT.jar.

Screenshot 5: gradle_nexus_publish.png



./gradlew clean build publish ran the Nexus publish task and finished with BUILD SUCCESSFUL.

Screenshot 6: nexus_gradle_artifact.png

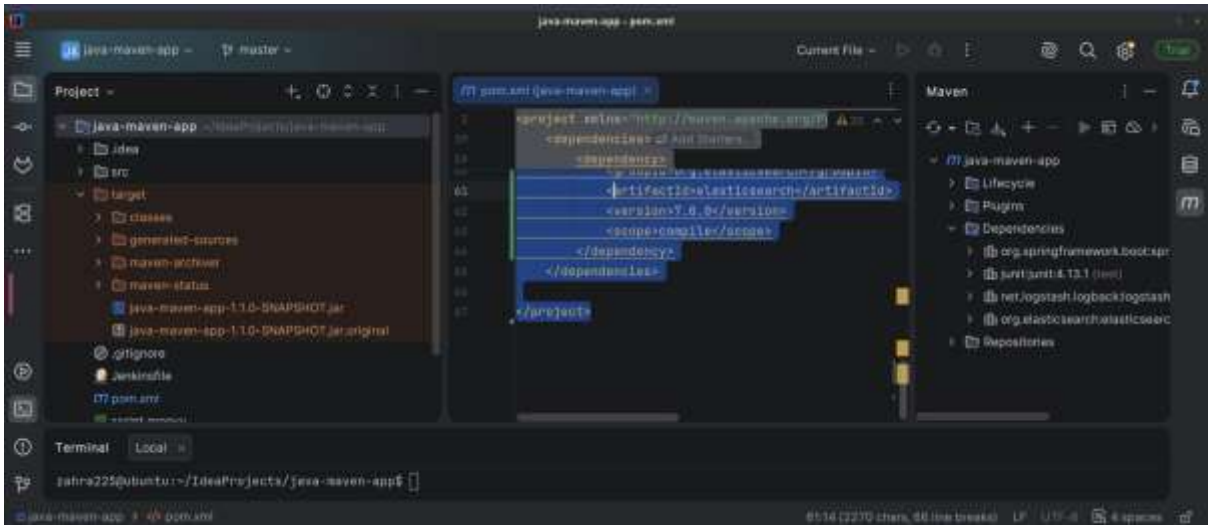


maven-snapshots in Nexus showing com/example/my-app/1.0-SNAPSHOT with the published JAR and metadata.

Task 4: Publish the Maven Application to Nexus

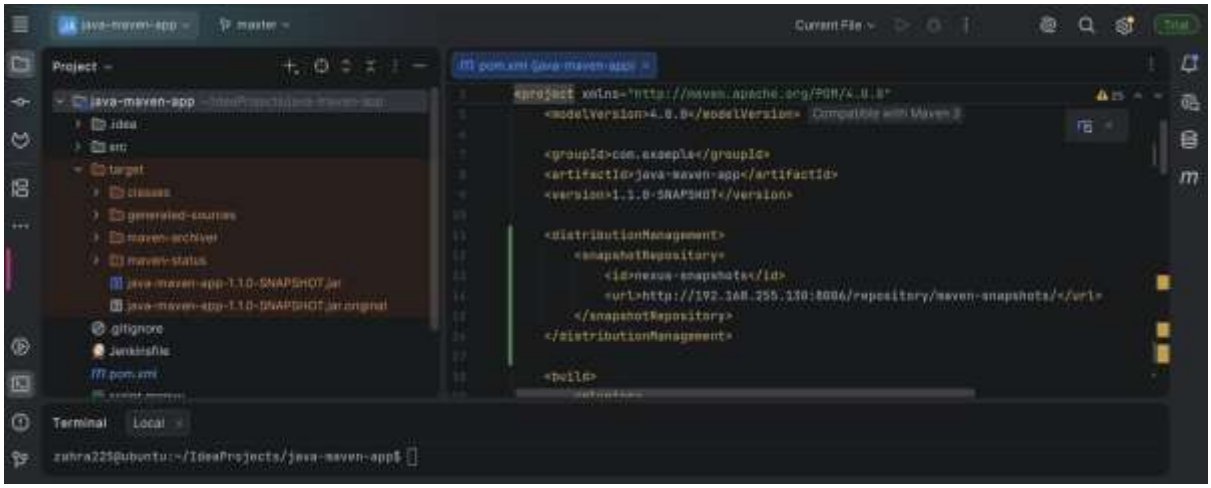
I opened the java-maven-app project, added the distributionManagement snapshot repository to pom.xml and configured the matching nexus-snapshots server credentials in ~/.m2/settings.xml. I ran mvn package and mvn deploy and verified the snapshot artifact in Nexus.

Screenshot 7: maven_nexus_project.png



java-maven-app project opened in IntelliJ.

Screenshot 8: maven_distribution_management.png



distributionManagement block with the nexus-snapshots ID and maven-snapshots URL in pom.xml.

Screenshot 9: maven_package_nexus.png



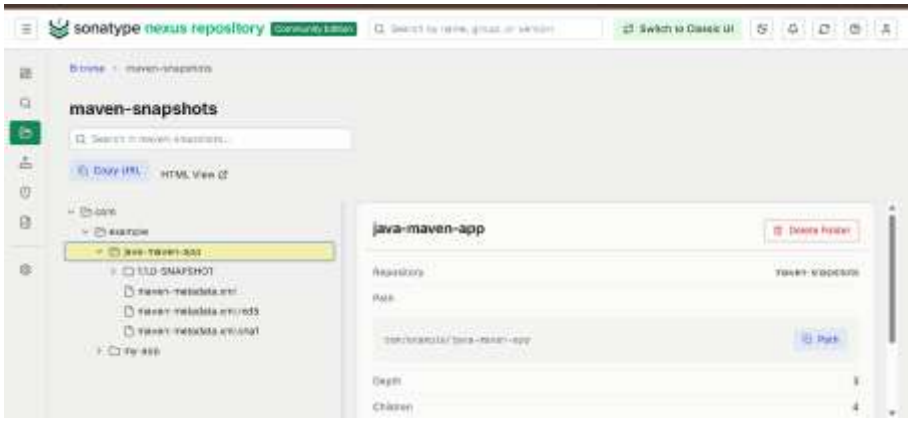
mvn package finished with BUILD SUCCESS and created the snapshot JAR in target.

Screenshot 10: maven_deploy_nexus.png



mvn deploy uploaded the artifact to Nexus and finished with BUILD SUCCESS.

Screenshot 11: nexus_maven_artifact.png



maven-snapshots in Nexus showing com/example/java-maven-app/1.1.0-SNAPSHOT with the JAR, POM and metadata.

Task 5: Inspect Nexus Repositories and Components Using the REST API

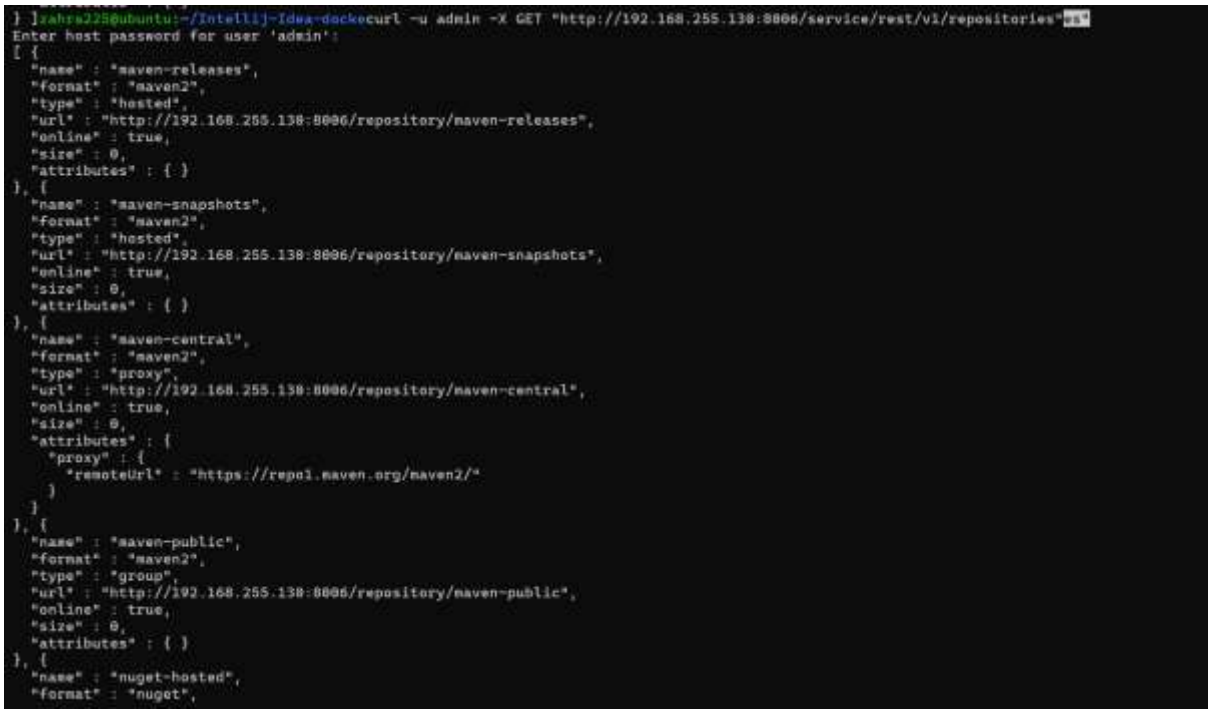
From the Ubuntu server terminal I used curl with the Nexus REST API. I listed the repositories as the nexus user and as admin, listed the components in maven-snapshots and retrieved one component by its ID. Passwords were entered at the curl prompt and are not visible.

Screenshot 12: nexus_repositories_user.png



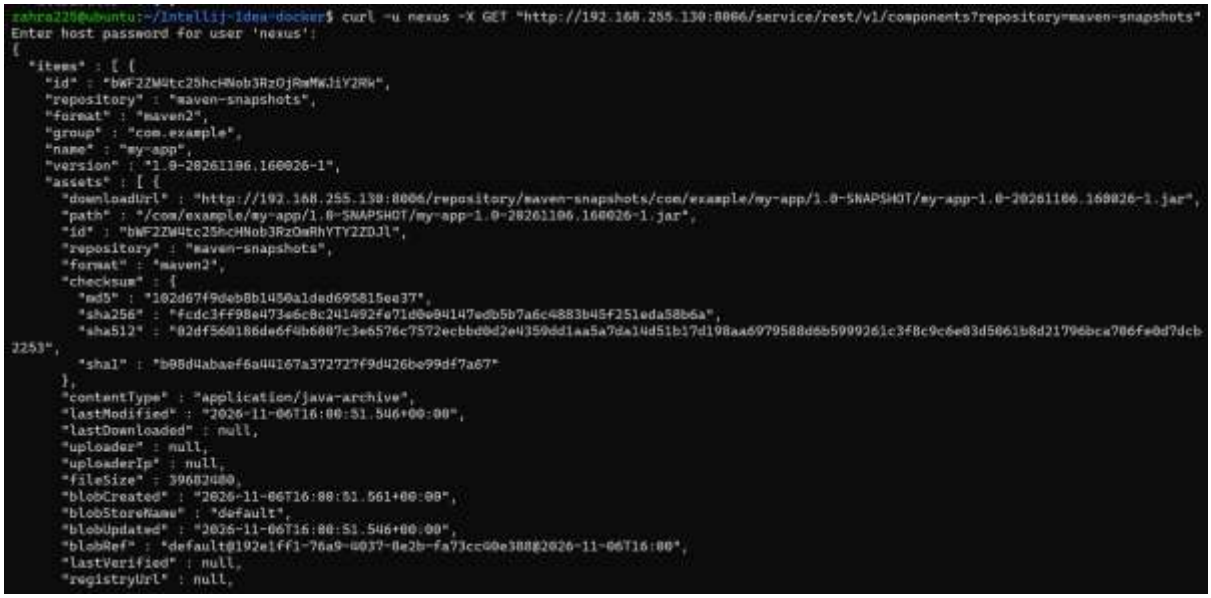
Repository list returned for the restricted nexus user.

Screenshot 13: nexus_repositories_admin.png



Repository list returned for the admin user.

Screenshot 14: nexus_components_list.png



Components in maven-snapshots with their id, repository, name and version.

Screenshot 15: nexus_component_details.png



Details of one component retrieved by its ID.

Task 6: Create a Nexus Blob Store and Docker Hosted Repository

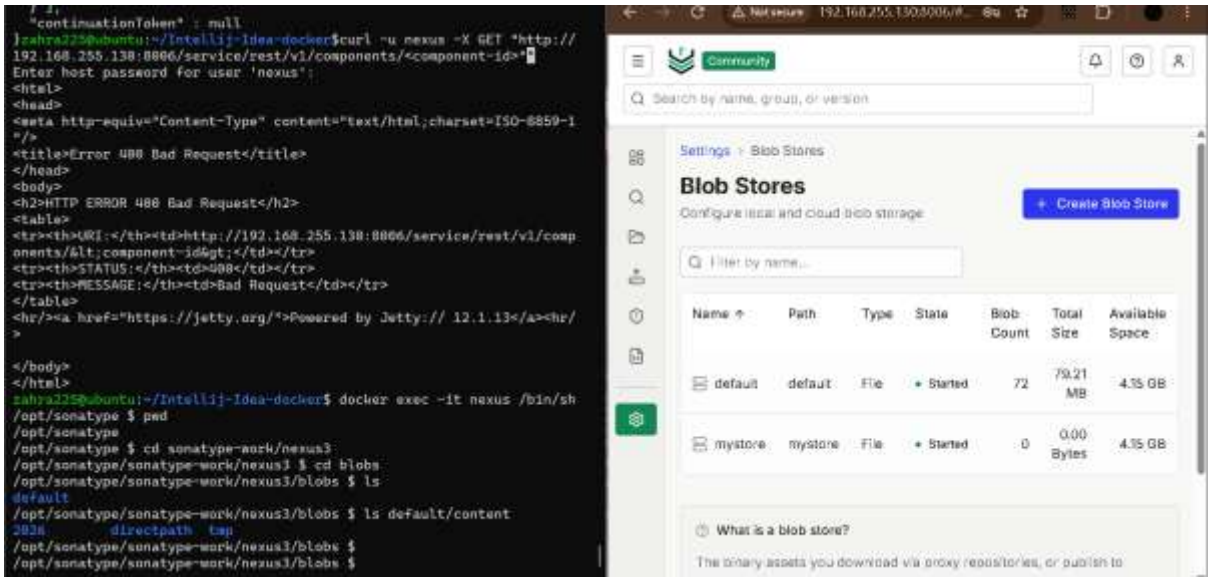
I inspected the default blob store directory inside the Nexus container without modifying it, created the mystore file blob store and verified its directory. I then created the docker-hosted repository on HTTP connector port 8007 using the mystore blob store.

Screenshot 16: nexus_default_blob_content.png

```
zahra225@ubuntu:~/Intellij-Idea-docker$ docker exec -it nexus /bin/sh
/opt/sonatype $ pwd
/opt/sonatype
/opt/sonatype $ cd sonatype-work/nexus3
/opt/sonatype/sonatype-work/nexus3 $ cd blobs
/opt/sonatype/sonatype-work/nexus3/blobs $ ls
default
/opt/sonatype/sonatype-work/nexus3/blobs $ ls default/content
2026        directpath  tmp
/opt/sonatype/sonatype-work/nexus3/blobs $
```

Shell inside the Nexus container showing the blobs directory, the default store and ls default/content.

Screenshot 17: nexus_mystore_created.png



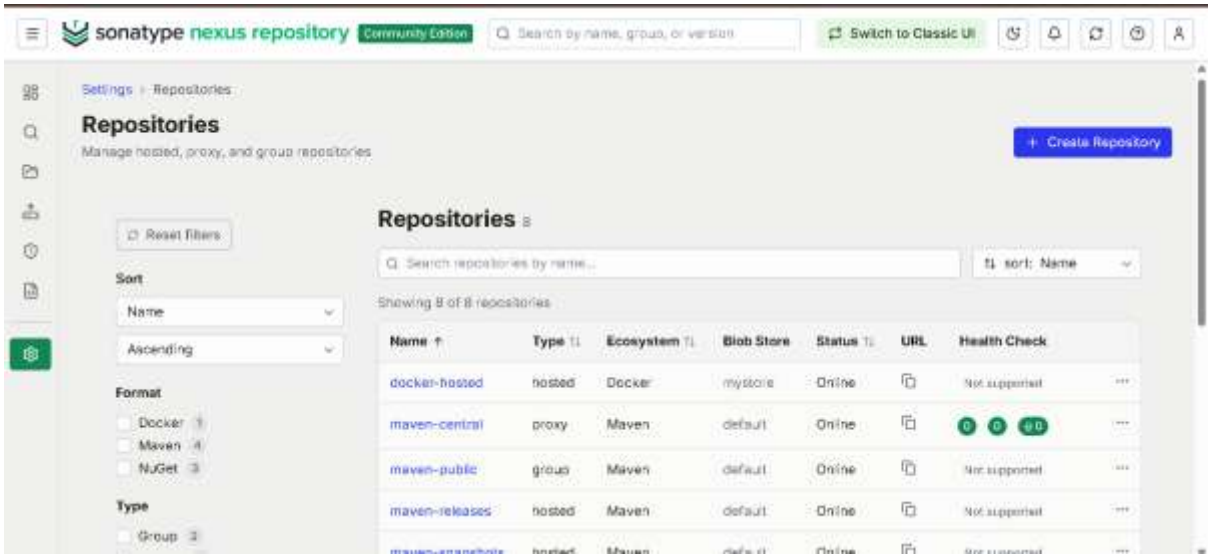
Blob Stores page showing default and mystore, both File type and Started.

Screenshot 18: nexus_blob_directories.png

```
/opt/sonatype/sonatype-work/nexus3/blobs $ ls
default
/opt/sonatype/sonatype-work/nexus3/blobs $ ls default/content
2026        directpath  tmp
/opt/sonatype/sonatype-work/nexus3/blobs $
/opt/sonatype/sonatype-work/nexus3/blobs $ pwd
/opt/sonatype/sonatype-work/nexus3/blobs
/opt/sonatype/sonatype-work/nexus3/blobs $ ls
default  mystore
/opt/sonatype/sonatype-work/nexus3/blobs $ ls mystore
content             metadata.properties
/opt/sonatype/sonatype-work/nexus3/blobs $ exit
zahra225@ubuntu:~/Intellij-Idea-docker$
```

blobs directory containing default and mystore, and the Ubuntu prompt after exit.

Screenshot 19: nexus_docker_hosted_repository.png

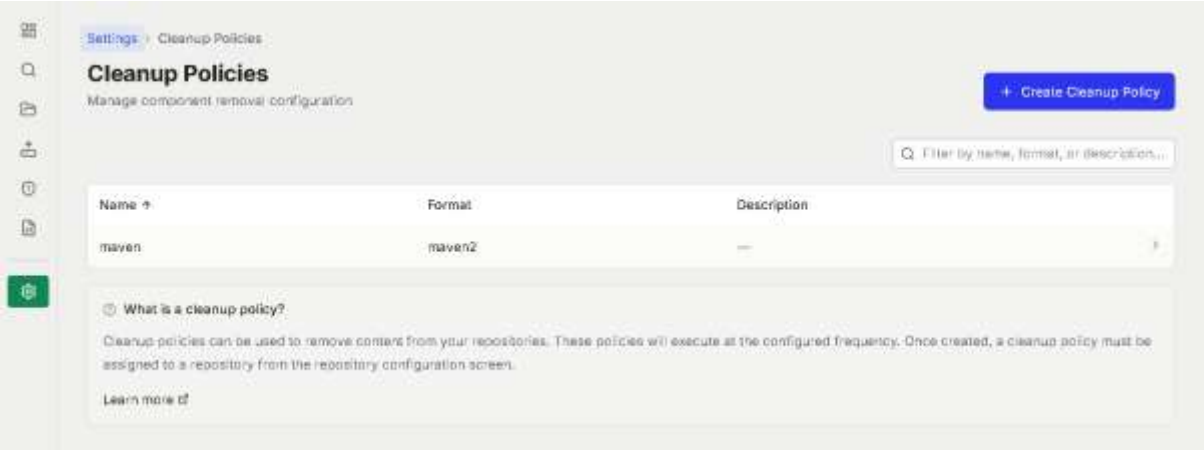


docker-hosted repository: Online, docker (hosted), blob store mystore, HTTP connector 8007.

Task 7: Create and Run a Maven Snapshot Cleanup Policy

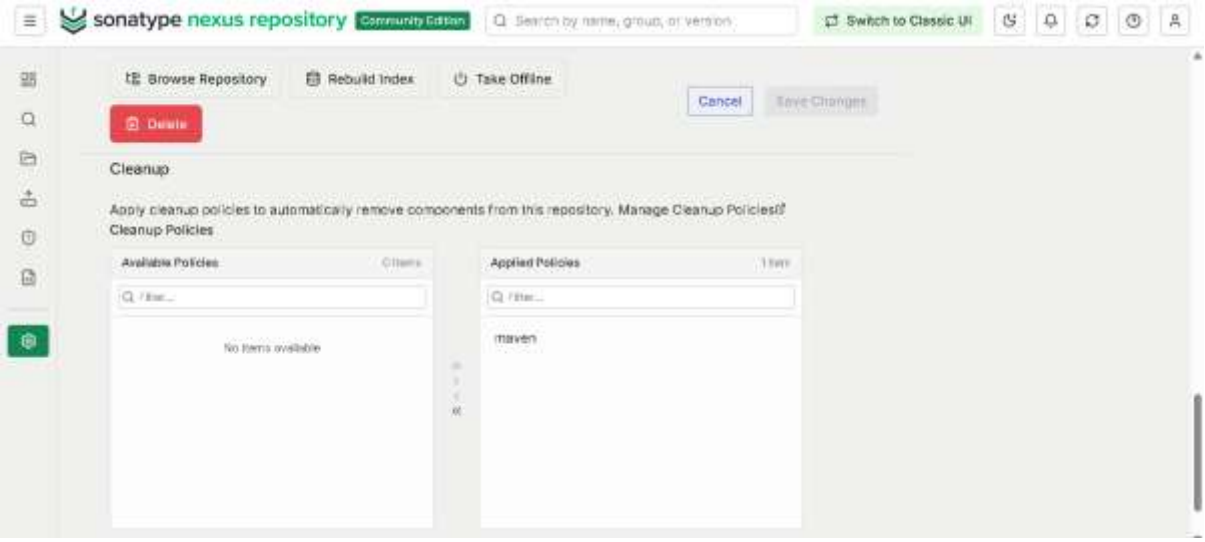
After saving all earlier evidence, I created the maven cleanup policy (maven2, Pre-Releases/Snapshots, asset name matcher .*), attached it to maven-snapshots and ran the Cleanup service manually. maven-snapshots became empty. I then created and ran the compact-store task on the default blob store and confirmed that disk space was reclaimed.

Screenshot 20: nexus_cleanup_policy_created.png



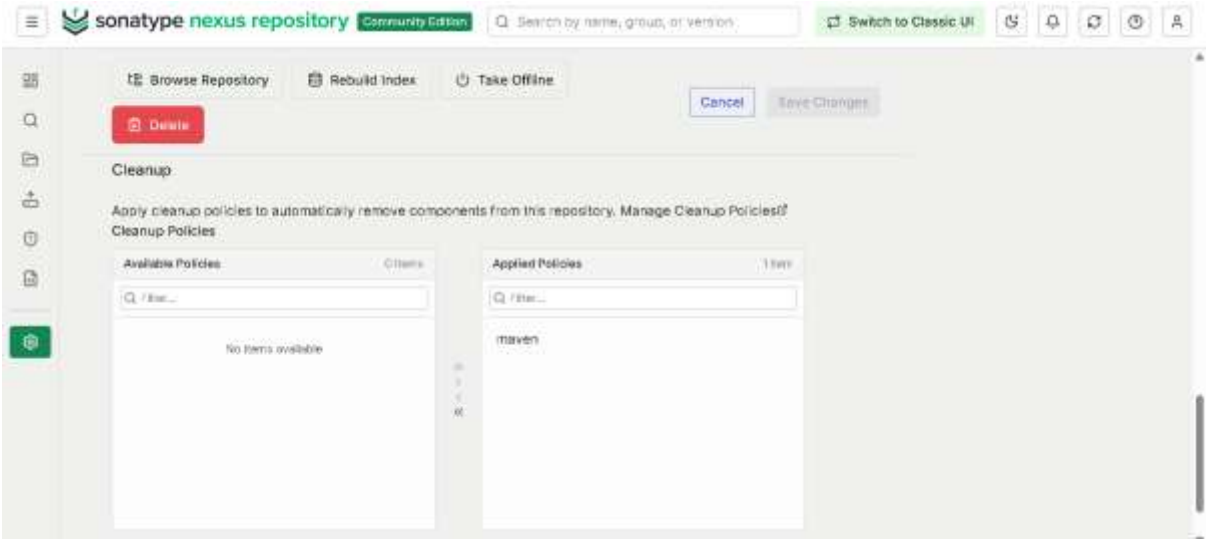
maven cleanup policy with the populated maven-snapshots preview.

Screenshot 21: nexus_cleanup_policy_attached.png



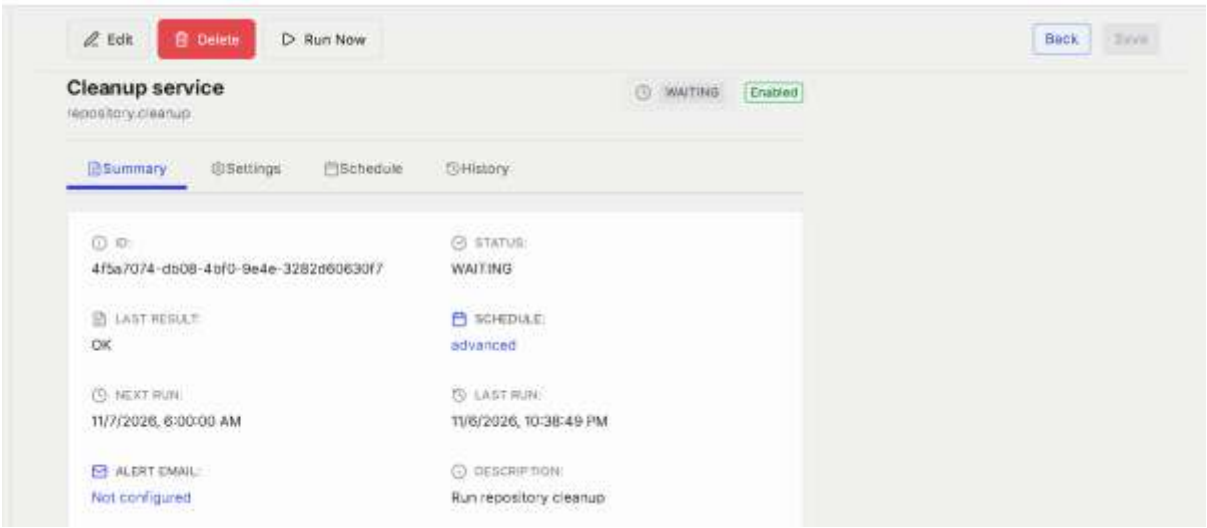
maven policy in the Applied list of the maven-snapshots repository.

Screenshot 22: nexus_cleanup_task_waiting.png



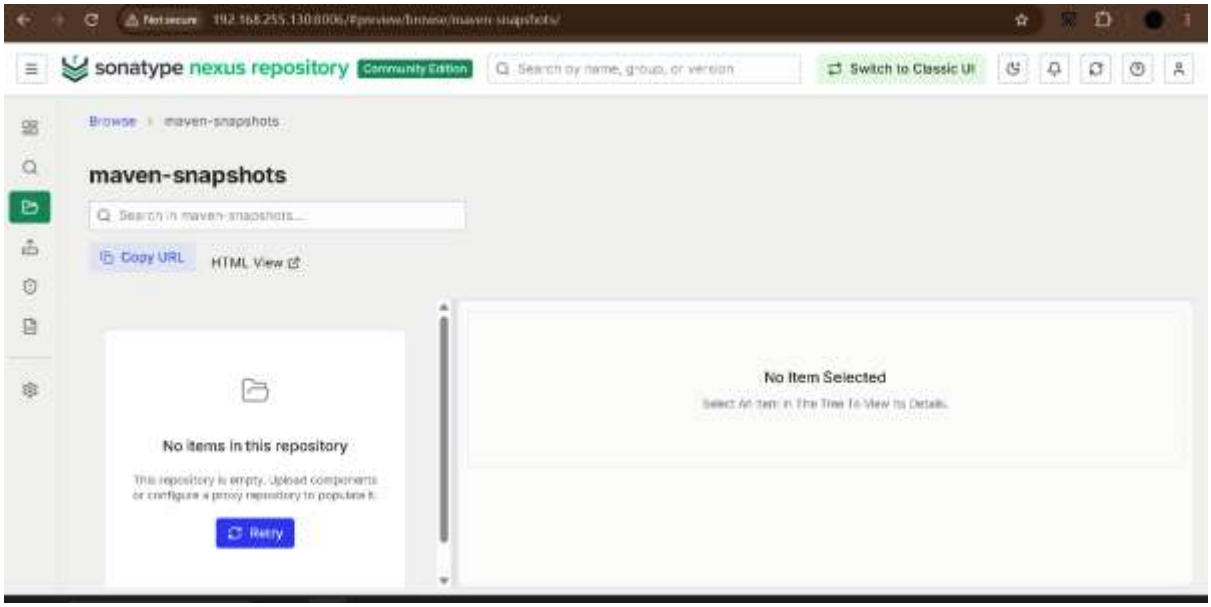
Cleanup service task in Waiting status with the next run at 6:00 AM.

Screenshot 23: nexus_cleanup_task_completed.png



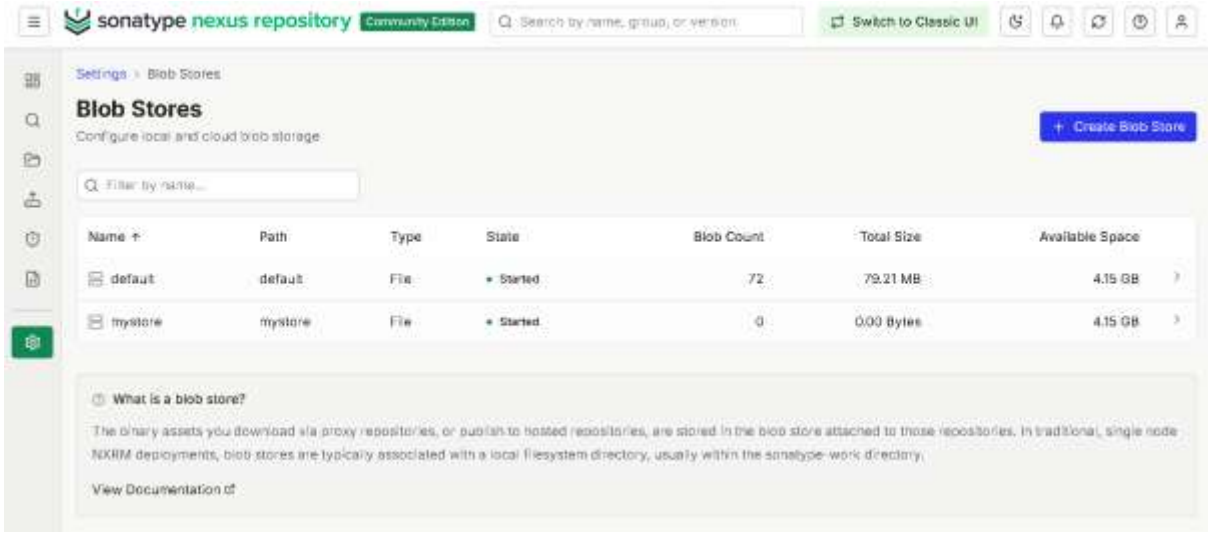
Cleanup service after the manual run: updated Last run and Last result Ok.

Screenshot 24: nexus_maven_snapshots_empty.png



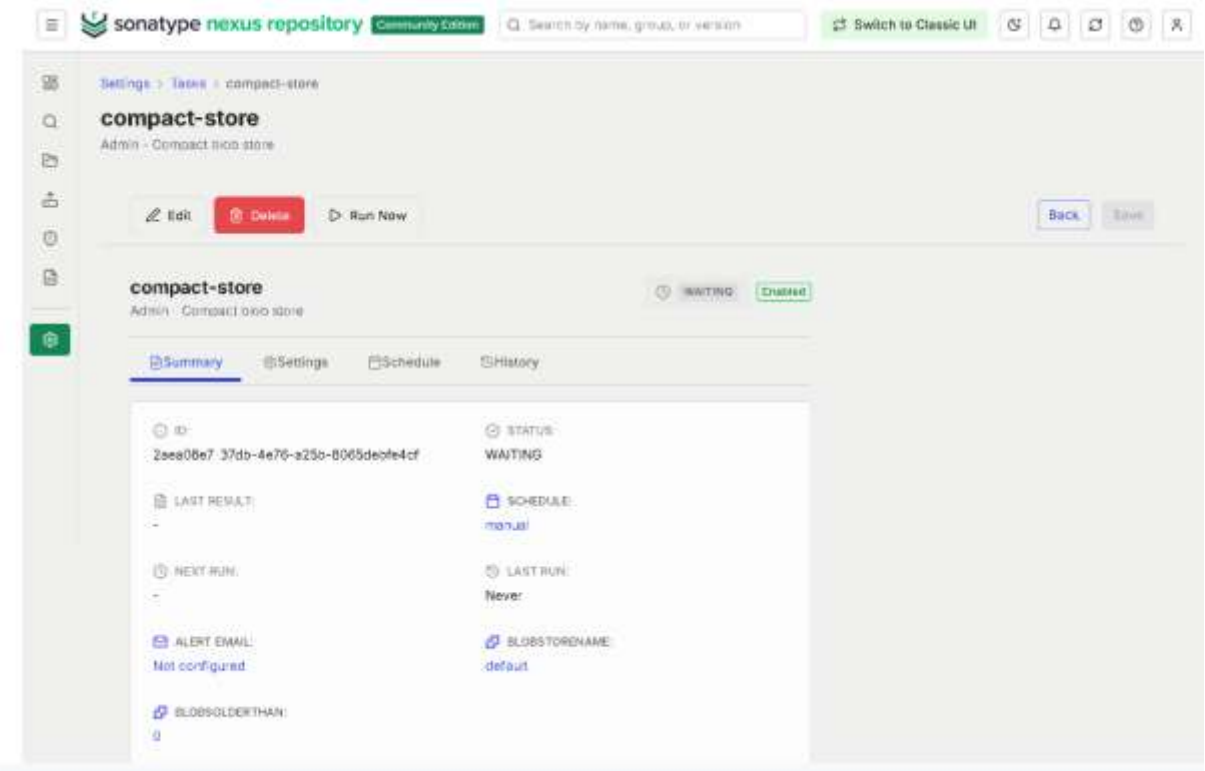
maven-snapshots shows: No component/assets found in repository.

Screenshot 25: nexus_blob_store_before_compaction.png



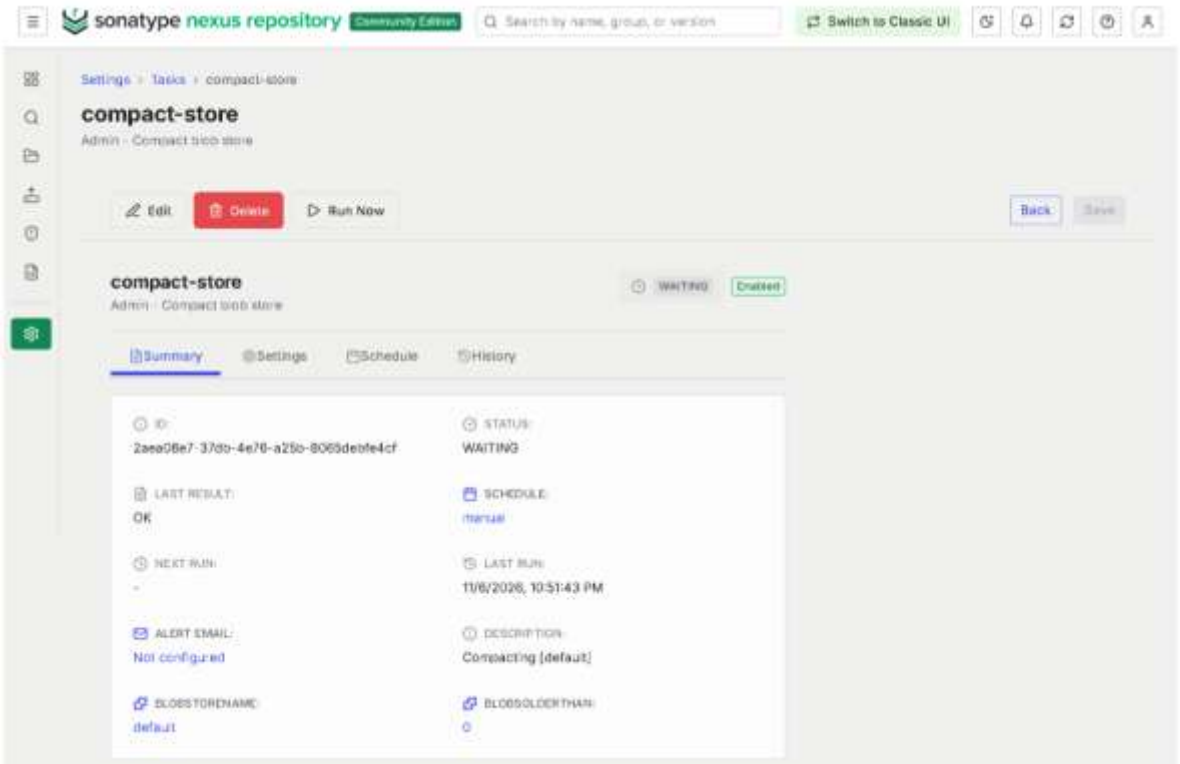
default blob store blob count, used size and available space before compaction.

Screenshot 26: nexus_compact_task_configured.png



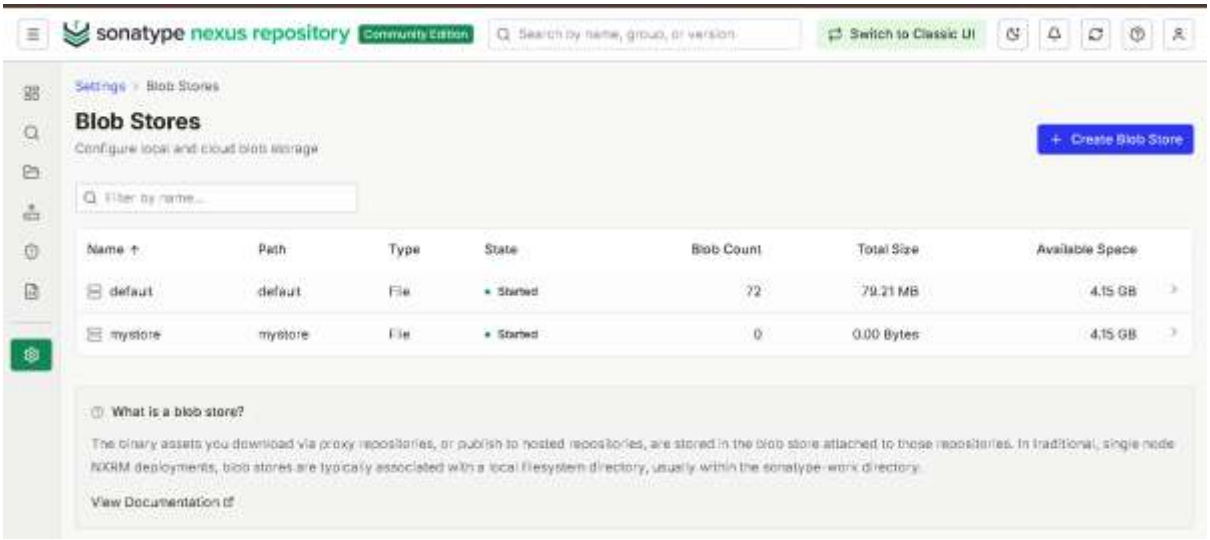
compact-store task: default blob store, 0 days, daily at 06:30.

Screenshot 27: nexus_compact_task_completed.png



compact-store after the manual run: updated Last run and Last result Ok.

Screenshot 28: nexus_blob_store_after_compaction.png



default blob store after compaction with lower used size and more available space.